

1. Présentation du Projet

Nom du projet : PharmaFinder

1. Énoncé du Problème

Aujourd'hui, les patients sont fréquemment confrontés à l'indisponibilité de certains médicaments en pharmacie. Ce problème entraîne :

- **Une perte de temps et d'énergie :** Les utilisateurs doivent se déplacer physiquement dans plusieurs pharmacies pour trouver leur traitement.
- **Une rupture de soins :** Le retard dans la prise de médicaments peut causer des problèmes de santé au patient.

PharmaFinder répond à ce besoin en centralisant l'information des stocks en temps réel pour offrir un accès rapide, transparent et efficace aux produits de santé.

2. Présentation du Concept

Concept : Une plateforme web "fullstack" (Angular/Spring Boot) permettant de localiser des médicaments en temps réel et de vérifier les disponibilités ainsi que les prix dans les pharmacies de proximité.

Objectif : Résoudre le problème des ruptures de stock critiques et optimiser le parcours de santé des utilisateurs en leur offrant une visibilité immédiate sur l'inventaire des officines partenaires.

2. Analyse des Besoins (Use Cases)

le système distingue trois intervenants :

Acteur	Cas d'Utilisation	Description
Visiteur (Public)	Consulter les stocks	Chercher un médicament et voir les pharmacies aux alentours.
Pharmacien	Gérer son inventaire	Se connecter et modifier le prix/quantité de ses produits en stock.
Administrateur	Gérer le référentiel	Ajouter de nouvelles molécules ou valider l'inscription d'une officine.

3. Spécifications Fonctionnelles Détaillées

A. Module Utilisateur

- **Moteur de Recherche Dynamique** : Recherche par nom commercial ou principe actif avec autocomplétion (Angular RxJS).
- **Fiche Pharmacie** : Affichage de l'adresse, contact, et statut de disponibilité (En stock / Stock faible / Épuisé).

B. Module Pharmacien

- **Gestion des Stocks (CRUD)** : Interface pour mettre à jour en temps réel les quantités et les prix de vente.

C. Module Administrateur

- **Référentiel Médical** : Gestion de la base de données centrale
- **Modération** : Validation des comptes des pharmacies partenaires.

4. Spécifications Techniques (Stack)

4.1. Backend Java (Spring Boot 3)

- **Architecture** : Modèle Controller-Service-Repository.
- **Spring Data JPA** : Gestion des relations complexes et des requêtes filtrées.

4.2. Frontend (Angular 16+)

5. Flux de l'Application (User Flows)

Cette section détaille la communication entre le Frontend (Angular) et le Backend (Spring Boot).

5.1. Flux de Recherche et Consultation (Utilisateur)

Ce flux représente le parcours d'un utilisateur cherchant un médicament.

1. **Saisie** : L'utilisateur saisit le nom d'un médicament dans le composant de recherche Angular.
2. **Appel API** : Angular intercepte l'événement et envoie une requête HTTP GET vers l'endpoint `/api/stocks/search?name=nom_du_medicament`.
3. **Traitement Backend** : Le Controller Spring Boot reçoit la requête.
 - Le Service appelle le Repository JPA qui exécute une requête avec jointure (JOIN) entre les tables **Medicament**, **Stock** et **Pharmacie**.
 - Le moteur de base de données filtre les pharmacies ayant une quantité supérieure à zéro.
4. **Réponse** : Le serveur renvoie une liste d'objets DTO (Data Transfer Objects) contenant le nom de la pharmacie, l'adresse, le prix et la disponibilité.
5. **Affichage** : Angular reçoit le JSON et met à jour l'interface via un "pipe" de tri pour afficher les prix les plus bas en premier.

5.2. Flux de Mise à jour d'Inventaire (Pharmacien)

Ce flux sécurisé permet à une pharmacie de gérer son stock.

1. **Modification** : Le pharmacien connecté modifie la quantité ou le prix d'un produit sur son tableau de bord.
2. **Envoi** : Angular valide que la saisie est un nombre positif et envoie une requête HTTP PATCH vers : `/api/stocks/{id_stock}`.
3. **Confirmation** : Le backend renvoie un code de succès 200 OK. L'interface Angular affiche une notification de succès temporaire.

5.3. Flux d'Administration (Référentiel)

Ce flux assure la cohérence des données globales.

1. **Action** : L'administrateur système ajoute un nouveau médicament validé au catalogue national.
2. **Transmission** : Angular envoie une requête HTTP POST vers </api/admin/medicaments> avec les informations du produit
3. **Validation** : Spring Boot utilise Hibernate Validator pour vérifier que le code CIP respecte le format standard.
4. **Stockage** : Le médicament est ajouté à la table globale, le rendant disponible pour toutes les pharmacies du réseau.